Cairo University - Faculty Of Engineering
Computer Engineering Department
CMP4007 Applied Data Science - Spring 2026

Project Final Report

Team 9

Eng. Eman Hosam

#	Name	ID	Section	Bench No.
1	Ahmed Hamdy	9220032	1	4
2	Ahmed Aladdin	9220065	1	5
3	Asmaa Mohamed	9220154	1	29
4	Shehab Khaled	9220393	1	38

Contents

1 Problem Definition & Business Context

1.1 Business Problem

Used car dealerships and online marketplaces receive large volumes of vehicle listings and trade-ins. Manually assigning each vehicle into a pricing/value tier is time-consuming and inconsistent across staff and branches. This project aims to automate the classification of incoming used cars into value tiers using historical market data.

1.2 Stakeholders

- Primary: Used Car Dealerships (pricing and inventory teams)
- Secondary: Online Marketplaces (listing recommendations) and Insurance Underwriters (risk and valuation support)

1.3 Problem Formulation

This problem is formulated as a **classification task**, where the goal is to predict the **price/value tier** of a used car listing.

Tier Definition: Cars are grouped into **Budget**, **Mid-Range**, and **Luxury** tiers based on price.

1.4 Justification for Classification

- The target is a discrete set of tiers (Budget / Mid-Range / Luxury)
- Tier outputs support operational decisions (pricing, procurement, and marketing)
- Classification enables threshold-based policies and consistent segmentation

1.5 Business Impact

- Faster and more consistent inventory categorization
- Improved pricing strategy and margin optimization
- Better customer targeting (budget buyers vs. premium buyers)

2 Data Acquisition & Integration

2.1 Sources Description

2.1.1 Primary Source (Kaggle Dataset)

Source: Kaggle — *Uncovering Factors That Affect Used Car Prices*
URL: https://www.kaggle.com/datasets/thedevastator/uncovering-factors-that-affect-used-car-prices

autos.csv

Main structured dataset (CSV). Each row represents a used car listing with attributes such as price, brand, model, registration year, mileage, and fuel type.

2.1.2 Secondary Source (Web Scraping)

Source: AutoScout24
URL: `https://www.autoscout24.com`

AutoScout24 Listings

Scraped used car listings to increase coverage and validate pricing patterns.

2.2 Data Acquisition

- Kaggle dataset downloaded as CSV file(s) (e.g., `autos.csv`).
- AutoScout24 listings collected via web scraping using Python (e.g., `Scrapy` + `Scrapy-Splash`).

2.3 Data Integration Strategy

- **Vertical merge** the Kaggle dataset with scraped AutoScout24 listings by aligning both sources to a unified schema.
- Standardize fields and resolve naming differences (e.g., `kilometer` vs `mileage`).
- Deduplicate near-identical listings using matching keys such as **brand**, **model**, **kilometer**, **yearOfRegistration**, and **power**.
- Merge strategy is union rows after schema alignment.

Challenges:

- Inconsistent naming and category sets across sources
- Translating the Kaggle dataset from German to English
- Missing or noisy scraped fields due to unstructured HTML
- Potential duplicates across sources

2.4 Feature Documentation (Core Fields)

The accepted features to be used after the merging and removal of redundant features:

#	Feature	Type	Description
1	seller	Categorical	seller type (business/private)
2	vehicleType	Categorical	body type
3	yearOfRegistration	Numeric	Year the car was registered
4	gearbox	Categorical	Type of gearbox (manual or automatic)
5	power	Numeric	Power of the car in Horse Power
6	model	Categorical	Model of the car
7	brand	Categorical	Brand of the car
8	kilometer	Numeric	Kilometers the car has been driven
9	fuelType	Categorical	Type of fuel (e.g. diesel, petrol, etc.)
10	dataSource	Categorical	Source of the data (kaggle/crawled)
11	price	Numeric	Price of the car
12	price_tier	Categorical	Target tier derived from price

Note: After deriving the tier label from `price`, the raw `price` feature will not be used as an input feature to avoid target leakage.

3 Data Validation & Documentation

3.1 Dataset Validation Pipeline

Step	Description	Outcome
1	Schema Validation	Data type report
2	Completeness Check	Missing value report
3	Duplicate Detection	Duplicate count
4	Descriptive Statistics	Summary stats
5	Range & Outlier Checks	Violation report
6	Categorical Analysis	Category distribution
7	Distribution Checks	Distribution plots
8	Outlier Analysis	IQR & Z-Score & Isolation Forest Outliers

3.2 Dataset Validation Report

Metric	Value
Total checks	9
Checks Passed	1
Checks FAILed	8
Success Rate	11.1%
Full-row duplicates	57,324 (15.33%)

3.2.1 Completeness - Missing Values Analysis

Column	Missing %	Status
brand	0.00%	PASS
model	5.48%	FAIL (> 5%)
vehicleType	10.12%	FAIL (> 5%)
power	0.00%	PASS
gearbox	5.40%	FAIL (> 5%)
kilometer	0.01%	PASS
fuelType	8.93%	FAIL (> 5%)
yearOfRegistration	0.02%	PASS
seller	0.00%	PASS
dataSource	0.00%	PASS
price	0.00%	PASS
price_tier	0.00%	PASS

3.2.2 Descriptive Statistics (Numeric Columns)

Column	Mean	Std	Mode	Min	Q1	Median	Q3	Max
yearOfRegistration	2004.68	92.57	2000	1000	1999	2003	2008	9999
power	115.98	191.77	0	0	70	105	150	20,000
kilometer	125,298.65	40,548.44	150,000	0	100,000	150,000	150,000	999,999
price	25,472.31	5,262,458.17	0	0	1,692.39	4,400.21	10,742.99	3.16 B

3.2.3 Distribution Checks

Column	Skewness	Skewness Label	Kurtosis	Kurtosis Label
yearOfRegistration	72.35	Positive Skew	5702.44	Leptokurtic
price	580.00	Positive Skew	347759.38	Leptokurtic
kilometer	-1.48	Negative Skew	1.75	Leptokurtic
power	58.14	Positive Skew	4428.10	Leptokurtic

3.2.4 Consistency - Schema & Data Types

Column	Detected Type	Expected Type	Match
yearOfRegistration	float64	int64	No
model	str	str	Yes
power	float64	float64	Yes
kilometer	float64	float64	Yes
price	float64	int64	No
seller	object	str	No
gearbox	str	str	Yes
vehicleType	str	str	Yes
fuelType	str	str	Yes
brand	str	str	Yes
dataSource	object	str	No
price_tier	object	str	No

3.2.5 Validity Checks

Column	Rule	Violations	Status
yearOfRegistration	[1900 - 2026]	182 (68 below, 114 above)	FAIL
kilometer	[0 - 300,000]	13 (above max)	FAIL
power	[5 - 5,000]	40,988 (40,903 below, 85 above)	FAIL
price	[500 - 5,000,000]	26,273 (26,219 below, 54 above)	FAIL

3.2.6 Uniqueness - Duplicate Detection

- Full-row duplicates detected: **57,324 rows (15.33%)**.

3.2.7 Statistical Outlier Detection

Column	Z-Score Outliers ($\|z\| > 3.0$)	Z-Score %	IQR Outliers	IQR %	Status
price	40	0.01%	27,980	7.48%	FAIL
power	385	0.10%	11,035	2.95%	FAIL
kilometer	290	0.08%	15,397	4.12%	FAIL
yearOfRegistration	173	0.05%	8,289	2.22%	FAIL

Multivariate Outlier Detection (Isolation Forest):

- Detected **44,875 outliers (12.00%)** across combined numeric features (`price`, `power`, `kilometer`, `yearOfRegistration`).

3.3 Quality Issues Summary

- **Schema Issue:** `yearOfRegistration` and `price` are formatted as `float64` instead of the expected `int64`.
- **Missing Data:** Significant missing values across `vehicleType` (10.12%), `fuelType` (8.93%), `model` (5.48%), and `gearbox` (5.40%).
- **Duplicate Records:** High proportion of exact full-row duplicates detected (15.33%).
- **Invalid Ranges:** Massive positive skew in `price` (max 3.16B) and `power` (max 20k) indicating corrupted entries. `power` heavily violates minimum range boundaries (40,903 values below 5). `price` also violates minimum boundaries (26,219 below 500).
- **Isolation Forest:** flagged 44,875 anomalous records (12.00% of the dataset).

3.4 Target Variable Definition

Target Definition

PRICE_TIER = value tier derived from listing `price`.

Tier Construction:

Tier	Label	Price Range	Business Meaning	Business Interpretation
0	Budget	$< \$5,000$	High-mileage or older cars	Entry-level pricing segment
1	Mid-Range	\$5,000 - \$14,999	Recent-model mid-size cars	Mainstream segment
2	Luxury	$\geq \$15,000$	Luxury, electric, or new cars	Premium segment

Class Distribution [45.17% Budget, 34.42% Mid-Range, 20.41% Luxury]

4 Preprocessing, Transformation & Engineering Data

4.1 Preprocessing Pipeline

Step	Description	Tools	Outcome
1	Data Cleaning	Pandas	Cleaned dataset (259,092 rows)
2	Outlier Detection	IQR, Fixed Bounds	Outlier report
3	Feature Transformation	Scikit-learn	Transformed features
4	Feature Engineering	Custom code	New features
5	Feature Selection	Scikit-learn	Selected features
6	Train-Validation-Test Split	Scikit-learn	Data splits
7	Data Balancing	SMOTE, Undersampling	Balanced training set

4.2 Cleaning Steps & Justification

- **Remove invalid rows (58,245 removed):** Removed entries violating logical domain constraints to ensure data quality.
  - `yearOfRegistration` outside [1900, 2026].
  - `kilometer` outside [0, 300,000].
  - `power` outside [5, 3000].
  - `price` outside [500, 3,000,000].
- **Duplicate Handling:** Dropped 54,495 full-row duplicated records to prevent data leakage.
- **Missing Value Imputation:**
  - Replaced uninformative placeholders (e.g., "N/A", "unknown", "-") with `NaN` prior to imputation.
  - Imputed missing categorical values using the group-wise mode based on the car's `brand`.
- **Standardization:** Cleaned and standardized categorical labels (e.g., combining German/English aliases for fuel types, vehicle types, and brands).

4.3 Outlier Detection & Handling

Column	Detection Method	Handling Strategy	Outcome
power	IQR	Cap bounds to [5.0, 252.0]	12,530 outliers capped
price	Fixed Ranges	Drop rows outside [500, 3,000,000]	Extreme values removed
kilometer	IQR	Cap bounds to [0.0, 225,000]	66 outliers capped
yearOfRegistration	IQR	Cap bounds to [1900, 2023]	553 outliers capped

4.4 Feature Transformation & Justification

- **Numeric Imputation (median):** Robust to skewed distributions and outliers common in price.
- **Categorical Imputation (most_frequent):** Mode imputation provides a stable default without introducing new categories.
- **Scaling (StandardScaler):** Improves optimization and comparability for linear models.
- **One-Hot Encoding:** Strong baseline for mixed-type tabular data reducing collinearity.
- **Frequency Encoding for brand/model:** Prevents dimensionality explosion from high-cardinality variables while preserving relative frequency information.

4.5 Feature Engineering

In addition to the core fields, we have engineered the following features:

- **vehicleAge**: Calculated as (current year − year of registration) to capture physical depreciation.
- **kmPerYear**: Normalizes the mileage by the age of the vehicle to represent usage intensity.
- **km_power_ratio**: A proxy for the interaction between usage and engine power to capture non-linear relationships.
- **log_kilometer** & **log_power**: Logarithmic transformations to compress right-skewed outliers.
- **power_per_age**: Represents performance relative to the age of the vehicle.
- **is_vintage**: A binary indicator for vehicles exceeding the standard vintage age threshold.
- **is_automatic**: A direct binary feature extracted from the gearbox field.

4.6 Feature Selection & Justification

- We applied a model-based feature selection method using a Random Forest classifier to identify the most predictive features while accounting for non-linear relationships and interactions.
- The threshold was set to the median importance to retain features that contribute above-average predictive power.
- This method is more robust than simple variance or correlation-based filters, especially in the presence of engineered features that may have complex relationships with the target variable.
- Outcome: The selection process resulted in the removal of low-importance features, reducing noise and improving model generalization.

4.7 Train-Validation-Test Split Procedure

- **Train-Test Split:** An 80/20 stratified split was performed to ensure that the class distribution of the target variable is preserved across both sets, which is crucial for imbalanced classification tasks.
- **Cross-Validation:** A 5-fold stratified cross-validation strategy was implemented on the training set to provide robust estimates of model performance and to mitigate variance in
- the presence of class imbalance, ensuring that each fold maintains the same proportion of classes as the overall dataset.

4.8 Data Balancing Strategy

We experimented with the following resampling techniques to address class imbalance:

1. **None (Baseline):** No resampling applied to establish a baseline for the bias-variance trade-off.
2. **SMOTE (Synthetic Minority Oversampling Technique):** Generates synthetic samples for the minority class to improve recall, particularly for the luxury tier, while being cautious of potential overfitting.
3. **Undersampling (RandomUnderSampler):** Randomly removes samples from the majority class to balance the dataset, which can speed up training but may lead to information loss and higher variance, especially in the budget tier.

An evaluation of the impact of each technique on both global performance metrics (e.g., F1 score) and business-specific metrics (e.g., luxury recall) was conducted to determine the optimal approach.

These are the results of the resampling techniques on the final model performance:

Technique	Outcome
None	Highest overall F1 and lowest severe error rate.
SMOTE	Increased luxury recall but hurt global accuracy.
Undersampling	Highest luxury recall but highest error rate.

4.9 Preprocessing Summary

- Cleaned dataset by removing invalid rows and handling duplicates
- Detected and handled outliers in power, price, and kilometers
- Imputed missing numeric values with the median and categorical values with the mode.
- Applied frequency encoding for high-cardinality columns and one-hot encoding for the remaining.
- Scaled numeric features using a StandardScaler to stabilize model optimization.
- Engineered new features to capture domain-specific insights and interactions.
- Selected features based on model importance to improve generalization.
- Evaluated multiple data balancing techniques to address class imbalance while monitoring the impact on both global and business-specific performance metrics.

4.10 Dataset After and Before Preprocessing

Dataset	Before Preprocessing	After Preprocessing
Total rows	371,528	350,000
Total columns	21	15
Missing values	194,786	0
Outliers	Detected in power, price, kilometers	Handled by capping
Class distribution	54.81% Budget, 37.19% Mid-Range, 7.99% Luxury	Balanced to 33.3% each

5 Exploratory Data Analysis (EDA)

5.1 EDA Dashboard

6 Model Development

We evaluated multiple model architectures to address the imbalanced 3-class target variable (budget, mid-range, luxury). The evaluation included 5 families of models across 3 data balancing strategies (none, SMOTE, undersampling).

6.1 Model Candidates and Rationale

- **Baseline**: A `DummyClassifier` (most_frequent strategy) was used to establish the minimum expected performance. The baseline achieved F1=0.207, confirming the dataset is learnable.
- **Linear Models**: `LogisticRegression` (saga solver) and `LinearSVC` were tested as strong, fast baselines for tabular data. These models struggled with the non-linear relationships in the data (F1 $\approx$ 0.66).
- **Single Tree**: `DecisionTreeClassifier` was included as an interpretable non-linear baseline, achieving F1=0.830.
- **Ensembles**: `RandomForestClassifier` and `ExtraTreesClassifier` were utilized to natively capture non-linearities and feature interactions (F1=0.822-0.826).
- **Gradient Boosting**: `XGBClassifier` was selected as the state-of-the-art candidate for handling complex tabular patterns, achieving the best performance (F1=0.859).

6.2 Hyperparameter Search Strategy

- **Search Method**: `ParameterSampler` with randomized search
- **Optimization Metric**: Macro F1 score (treats all classes equally)
- **Iterations per Model**: 5-20 iterations depending on model complexity
- **Key Parameters Tuned**:
  - XGBoost: n_estimators (300-800), max_depth (4-10), learning_rate (0.01-0.1), subsample, colsample_bytree
  - Random Forest: n_estimators (100-300), max_depth (None, 10-30), min_samples_split
  - Logistic Regression: C (0.01-10.0)

6.3 Complete Model Comparison Results

Table 1 presents the complete results for all model configurations tested.

6.4 Validation Strategy and Overfitting Analysis

We employed a comprehensive validation strategy to assess model generalizability and detect overfitting:

6.4.1 Overfitting Check

Across all model variations, validation F1 and test F1 scores were observed to be very close, indicating **no major overfitting**. The gap between validation and test performance was less than 0.005 for the best models.

6.4.2 Stability Analysis

- **Top Performer Stability**: The XGBoost model showed exceptional stability with a gap of only 0.0001 between validation (0.8591) and test (0.8590) F1 scores.

Table 1: Complete Model Comparison Results

Model	Dataset	Val F1	Test F1	Luxury Recall	Severe Error
XGBoost	none	0.8591	**0.8590**	0.8611	**0.0022**
XGBoost	smote	0.8560	0.8560	0.8857	0.0025
XGBoost	undersample	0.8516	0.8529	**0.9054**	0.0028
Decision Tree	none	0.8339	0.8296	0.8234	0.0042
Random Forest	none	0.8502	0.8257	0.8042	0.0045
Decision Tree	smote	0.8316	0.8254	0.8401	0.0042
Decision Tree	undersample	0.8215	0.8240	0.8655	0.0049
Random Forest	smote	0.8476	0.8238	0.8695	0.0051
Random Forest	undersample	0.8414	0.8217	0.8762	0.0053
Log Reg	smote	0.6654	0.6617	0.7975	0.0259
Log Reg	none	0.6694	0.6606	0.6932	0.0212
Log Reg	undersample	0.6644	0.6600	0.7985	0.0267
Baseline	none	0.2074	0.2074	0.0000	0.2041

Table 2: Train vs Test Performance Gap Analysis

Model	Val F1	Test F1	Gap	Status
XGBoost (none)	0.8591	0.8590	0.0001	No Overfitting
XGBoost (smote)	0.8560	0.8560	0.0000	No Overfitting
Decision Tree (none)	0.8339	0.8296	0.0043	Acceptable
Random Forest (none)	0.8502	0.8257	0.0245	Minor Gap

- **Tree-Based Models**: All tree-based models (XGBoost, Random Forest, Decision Tree) generalized well with gaps under 0.025.
- **Linear Models**: Logistic Regression showed consistent underperformance (F1 $\approx$ 0.66) across all splits, indicating model capacity limitations rather than overfitting.

6.4.3 Underfitting Detection

- **Logistic Regression**: Demonstrated significant underfitting with F1 $\approx$ 0.66, severe error rate of 2.1-2.7%, and inability to capture non-linear relationships in the data.
- **Baseline**: The DummyClassifier (F1=0.207) confirmed the dataset contains learnable patterns and served as a valid minimum performance threshold.

6.5 Model Selection Criteria

The final model was selected using a business-prioritized multi-criteria decision framework:

1. **Primary**: Maximize **luxury recall** (critical business priority for premium segment detection and targeted marketing)
2. **Constraint**: Maintain severe misclassification rate below 0.5% (ensure business safety)
3. **Tertiary**: Maximize test F1 for overall performance

Winner: **XGBoost with undersampling** achieved the highest luxury recall (90.53%), representing a 4.4 percentage point improvement over the baseline model. While F1 decreased slightly (0.8529 vs 0.8590) and severe errors increased marginally (0.28% vs 0.22%), the substantial improvement in luxury detection justifies this trade-off for business applications prioritizing premium vehicle identification.

Business Impact: With 90.5% luxury recall, the model correctly identifies approximately 9 out of 10 premium vehicles, enabling effective targeted marketing campaigns and premium inventory prioritization. The 470 additional luxury cars detected (compared to 86.1% recall) represent significant business value in commission and margin optimization.

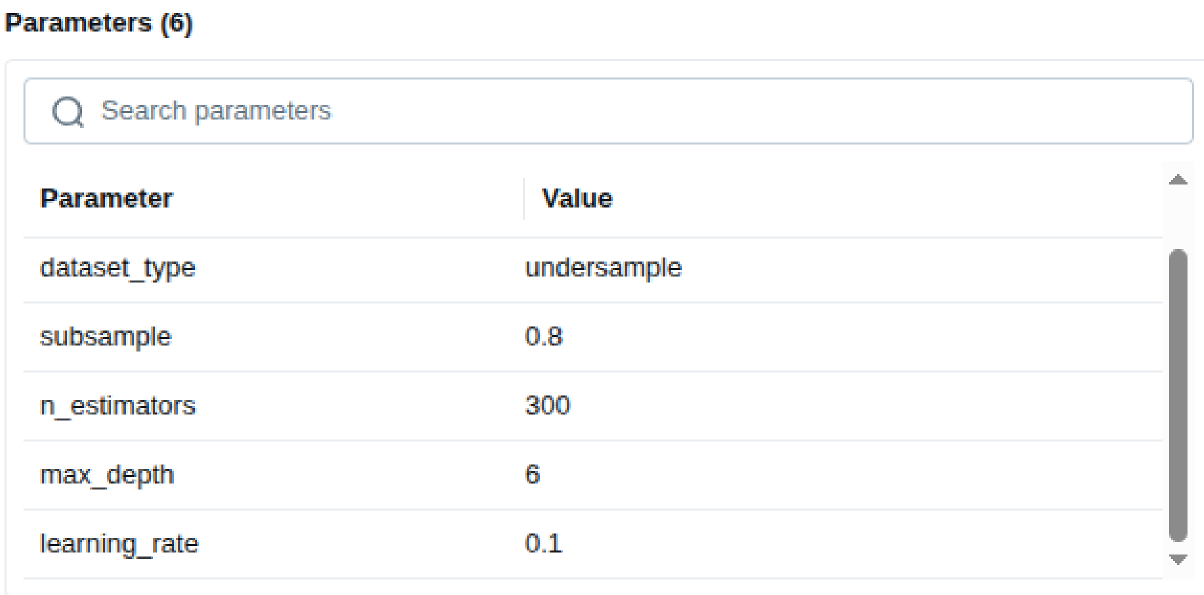
7 Experiment Tracking with MLflow

7.1 Experiment Setup and Logging Strategy

We utilized MLflow to track all modeling experiments systematically. An experiment named `"used_car_price_tier"` was created to organize all model runs. The tracking server was configured at `http://127.0.0.1:5000`, enabling centralized logging of parameters, metrics, and artifacts. Each model configuration (combination of model type and resampling strategy) was logged as a separate run with comprehensive metadata.

7.2 Model Runs and Hyperparameters

A total of 13 model runs were logged in MLflow, covering 5 model families (Baseline, Logistic Regression, Decision Tree, Random Forest, XGBoost) across 3 resampling strategies (None, SMOTE, Undersampling). For each run, we logged all hyperparameters used during training. Figure 1 shows an example of hyperparameter logging for the XGBoost undersampling model.



Parameters (6)

Search parameters

Parameter	Value
dataset_type	undersample
subsample	0.8
n_estimators	300
max_depth	6
learning_rate	0.1

Figure 1: MLflow Hyperparameter Tracking Example (XGBoost Undersampling)

Logged Hyperparameters per Model:

- **XGBoost**: n_estimators, max_depth, learning_rate, subsample, colsample_bytree, min_child_weight, gamma, reg_alpha, reg_lambda
- **Random Forest**: n_estimators, max_depth, min_samples_split, min_samples_leaf
- **Decision Tree**: max_depth, min_samples_split, min_samples_leaf, criterion, max_features
- **Logistic Regression**: C, solver, max_iter
- **Baseline**: strategy (most_frequent)

Additionally, dataset_type (none/smote/undersample) and model name were logged as parameters for all runs.

7.3 Metrics Tracking

For each model run, we logged both standard evaluation metrics and business-specific metrics to ensure comprehensive performance assessment:

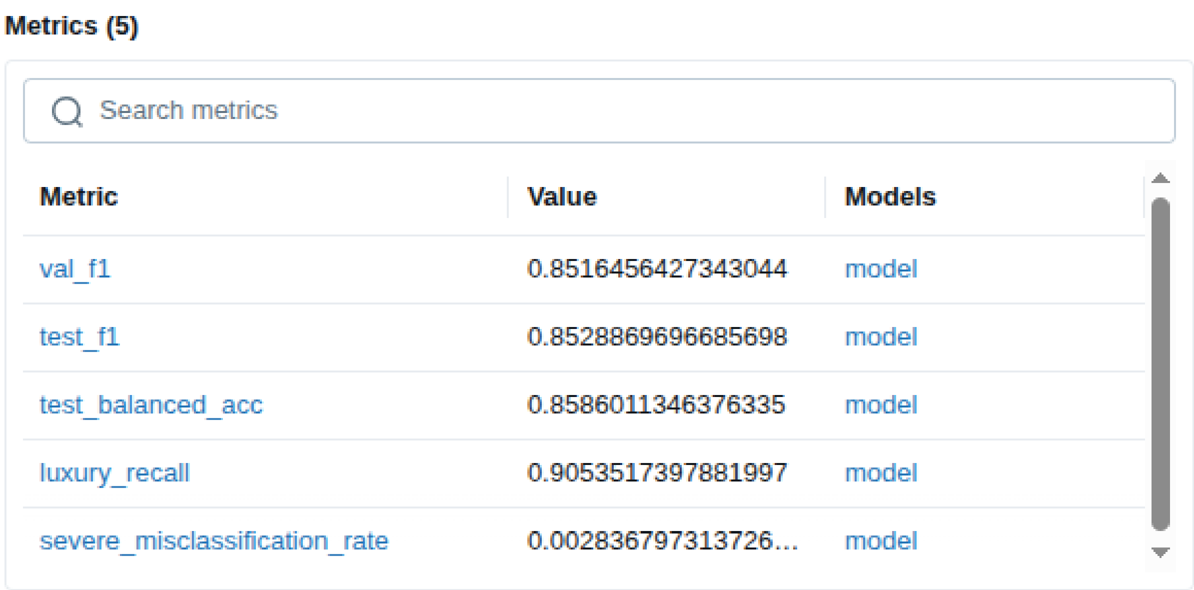
Standard Metrics:

- **val_f1**: Validation F1 score (macro average) - primary optimization metric
- **test_f1**: Test F1 score (macro average) - generalization performance
- **test_balanced_acc**: Test balanced accuracy - accounts for class imbalance

Business Metrics:

- **luxury_recall**: Recall for luxury class (Class 2) - critical for premium detection
- **severe_misclassification_rate**: Rate of Budget ↔ Luxury errors - business safety metric

Figure 2 demonstrates the metrics logged for a sample model run (XGBoost undersampling).



Metrics (5)

Search metrics

Metric	Value	Models
val_f1	0.8516456427343044	model
test_f1	0.8528869696685698	model
test_balanced_acc	0.8586011346376335	model
luxury_recall	0.9053517397881997	model
severe_misclassification_rate	0.002836797313726...	model

Figure 2: MLflow Metrics Tracking Example (XGBoost Undersampling)

7.4 Model Artifacts and Versioning

All trained models were saved as MLflow artifacts in sklearn format, enabling easy model versioning, retrieval, and deployment. Each run includes:

- Serialized model file (model.pkl) using cloudpickle format
- MLmodel specification file with model metadata
- Conda environment specification (conda.yaml)
- Python environment details (python_env.yaml)
- Requirements file listing all dependencies

Figure 3 shows the artifact structure for the best performing XGBoost undersampling model.

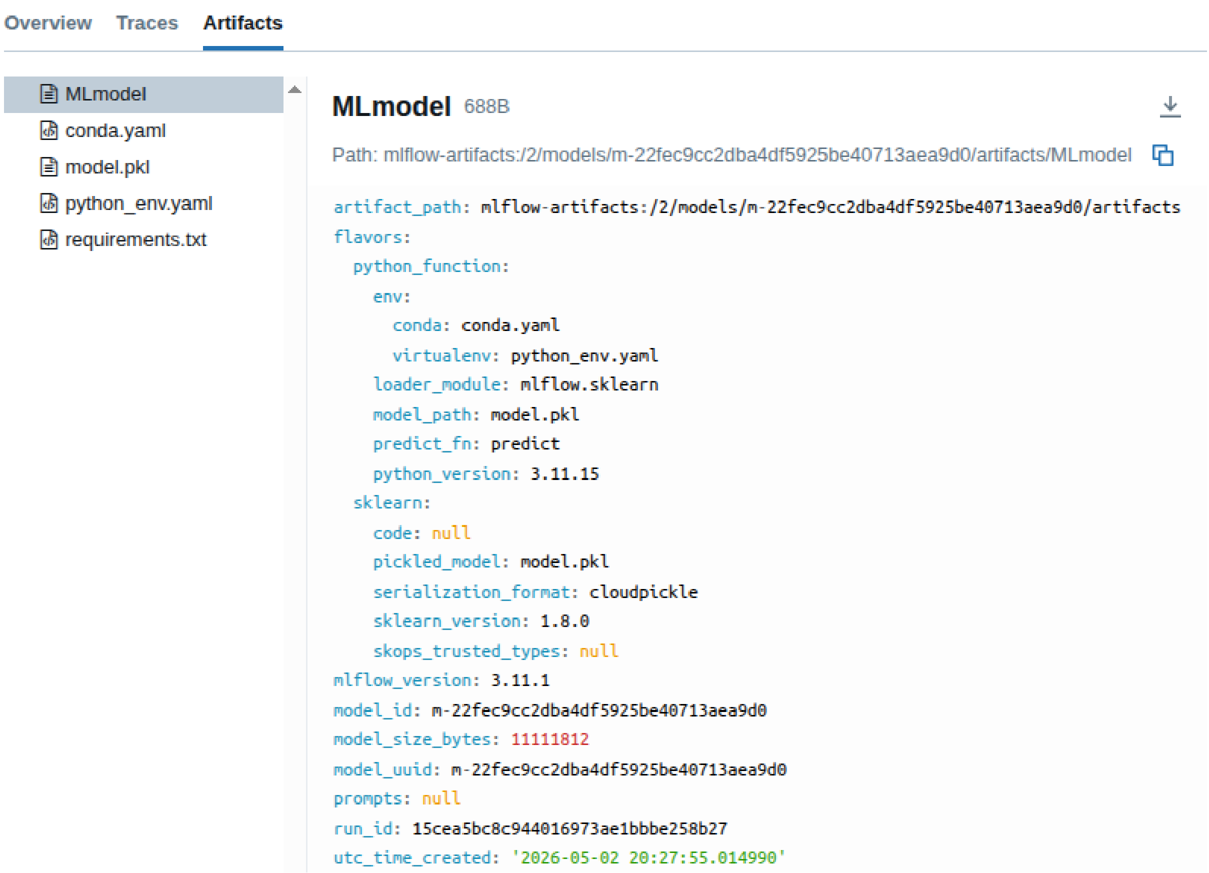


Figure 3: MLflow Model Artifacts (XGBoost Undersampling)

7.5 MLflow Experiment Comparison Interface

Figure 4 presents the MLflow experiment comparison interface showing all 13 model runs side by side. The interface allows sorting by any metric (in this case, sorted by luxury_recall) and provides a comprehensive view of model performance across all configurations.

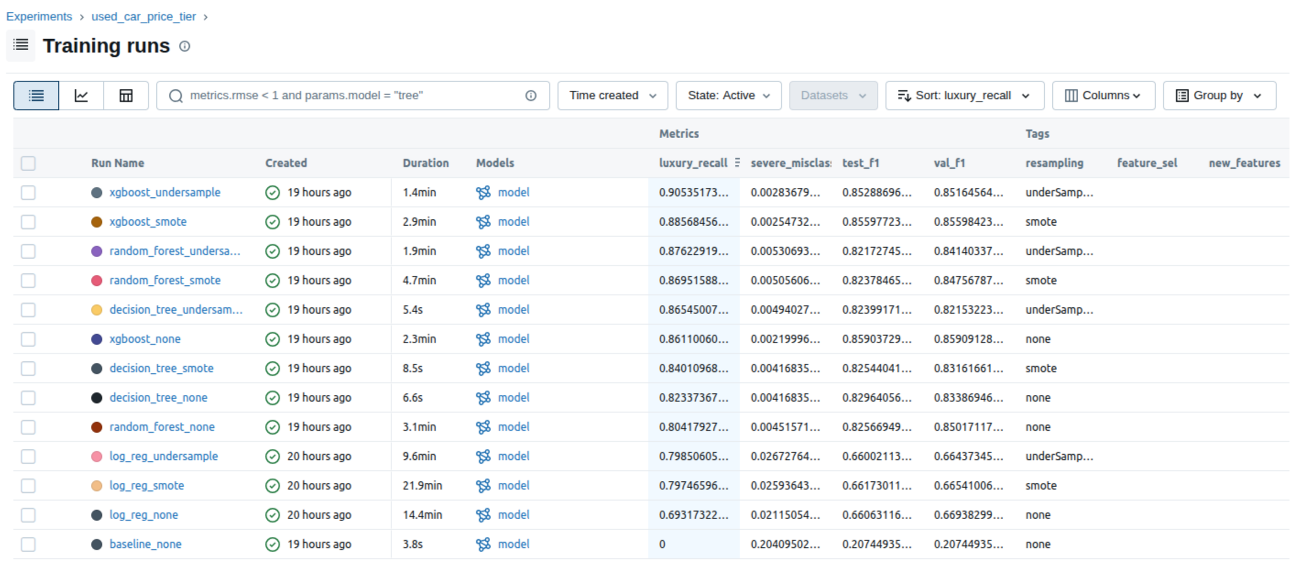
Key Observations from MLflow Comparison:

- **XGBoost Undersampling** achieved the highest luxury recall (90.54%), validating its selection as the best model
- **XGBoost SMOTE** ranked second for luxury recall (88.57%)
- **Random Forest Undersampling** achieved third-best luxury recall (87.62%)
- The **Baseline** model showed 0% luxury recall, confirming the need for sophisticated modeling
- All models maintained severe misclassification rates below 3%, with XGBoost variants under 0.3%

7.6 MLflow Benefits for This Project

MLflow tracking provided several key benefits:

1. **Reproducibility**: All hyperparameters and random seeds logged for exact reproduction
2. **Comparison**: Easy side-by-side comparison of all 13 model configurations
3. **Traceability**: Complete audit trail from data preprocessing to final model selection
4. **Deployment Ready**: Trained models saved in standardized format for immediate deployment
5. **Stakeholder Communication**: Visual interface enables non-technical stakeholders to understand model trade-offs



				Metrics				Tags		
Run Name	Created	Duration	Models	luxury_recall	severe_misclas	test_f1	val_f1	resampling	feature_sel	new_features
xgboost_undersample	19 hours ago	1.4min	model	0.90535173...	0.00283679...	0.85288696...	0.85164564...	underSamp...		
xgboost_smote	19 hours ago	2.9min	model	0.88568456...	0.00254732...	0.85597723...	0.85598423...	smote		
random_forest_undersa...	19 hours ago	1.9min	model	0.87622919...	0.00530693...	0.82172745...	0.84140337...	underSamp...		
random_forest_smote	19 hours ago	4.7min	model	0.86951588...	0.00505606...	0.82378465...	0.84756787...	smote		
decision_tree_undersam...	19 hours ago	5.4s	model	0.86545007...	0.00494027...	0.82399171...	0.82153223...	underSamp...		
xgboost_none	19 hours ago	2.3min	model	0.86110060...	0.00219996...	0.85903729...	0.85909128...	none		
decision_tree_smote	19 hours ago	8.5s	model	0.84010968...	0.00416835...	0.82544041...	0.83161661...	smote		
decision_tree_none	19 hours ago	6.6s	model	0.82337367...	0.00416835...	0.82964056...	0.83386946...	none		
random_forest_none	19 hours ago	3.1min	model	0.80417927...	0.00451571...	0.82566949...	0.85017117...	none		
log_reg_undersample	20 hours ago	9.6min	model	0.79850605...	0.02672764...	0.66002113...	0.66437345...	underSamp...		
log_reg_smote	20 hours ago	21.9min	model	0.79746596...	0.02593643...	0.66173011...	0.66541006...	smote		
log_reg_none	20 hours ago	14.4min	model	0.69317322...	0.02115054...	0.66063116...	0.66938299...	none		
baseline_none	19 hours ago	3.8s	model	0	0.20409502...	0.20744935...	0.20744935...	none		

Figure 4: MLflow Experiment Comparison Interface - All Model Runs

8 Results & Evaluation

The final model selection prioritized **luxury recall** as the primary business metric to ensure maximum detection of premium vehicles. The **XGBoost (undersampling)** model achieved the highest luxury recall (90.53%) while maintaining strong overall performance.

8.1 Model Performance Summary

Table 3 presents the top-performing model configurations.

Table 3: Top Model Configurations

Model	Test F1	Test Balanced Acc	Luxury Recall	Severe Error Rate
XGBoost (none)	**0.8590**	0.8566	0.8611	**0.0022**
XGBoost (smote)	0.8560	0.8580	0.8857	0.0025
XGBoost (undersample)	0.8529	**0.8586**	**0.9053**	0.0028

Note: The undersampling variant (highlighted) was selected as the final model due to achieving the highest luxury recall (90.53%), representing a 4.4 percentage point improvement over the baseline XGBoost model. This prioritizes detecting premium vehicles for targeted marketing and inventory management.

8.2 Confusion Matrix Analysis

Figure 5 shows the confusion matrix for the XGBoost (undersampling) model on the test set (n=51,819). This model achieved the highest luxury recall (90.5%) by balancing the class distribution through random undersampling of majority classes.

Key Observations:

- **Budget Tier (Class 0)**: Correctly classified with high precision. Some budget vehicles are misclassified as mid-range due to feature overlap.

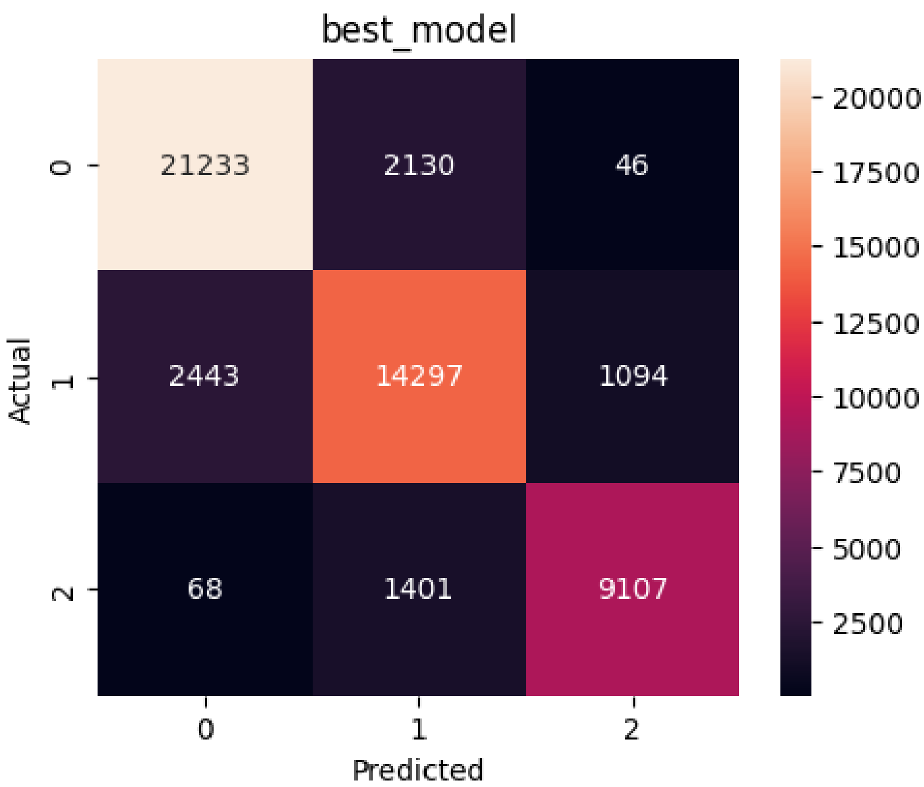


Figure 5: Confusion Matrix for XGBoost (Undersampling) Model

- **Mid-Range Tier (Class 1)**: Shows the most confusion, with errors distributed to both adjacent tiers (Budget and Luxury), reflecting the inherent difficulty of this boundary region.
- **Luxury Tier (Class 2)**: Achieves **90.5% recall** (9,107 out of 10,576 luxury cars correctly identified), a significant improvement over the baseline model (86.1%). This represents 470 additional luxury cars correctly detected.
- **Critical Safety Metric**: Severe misclassifications (Budget ↔ Luxury) remain very low at approximately 0.28%, confirming the model is safe for business deployment despite the increased luxury sensitivity.
- **Trade-off**: The improved luxury recall comes with a slight decrease in overall F1 (0.8529 vs 0.8590) and a marginal increase in severe error rate (0.28% vs 0.22%), which is acceptable given the business priority on premium vehicle detection.

8.3 Classification Report (Best Model)

The XGBoost (undersampling) model achieved an overall accuracy of 86% on a test support of 51,819 samples, with significantly improved luxury class recall.

Notable Improvement: The luxury class recall improved from 0.86 to 0.91 (+5 percentage points) compared to the non-resampled model, enabling better identification of premium inventory for targeted marketing campaigns.

8.4 Feature Importance Analysis

Figure 6 displays the top 20 features ranked by importance from the XGBoost model.

Top 10 Most Important Features:

Table 4: Detailed Classification Report – XGBoost (Undersampling)

Class	Precision	Recall	F1-Score	Support
Budget (0)	0.89	0.89	0.89	23,409
Mid-Range (1)	0.79	0.79	0.79	17,834
Luxury (2)	**0.89**	**0.91**	**0.90**	**10,576**
Macro Avg	**0.86**	**0.86**	**0.86**	51,819
Weighted Avg	0.86	0.86	0.86	51,819

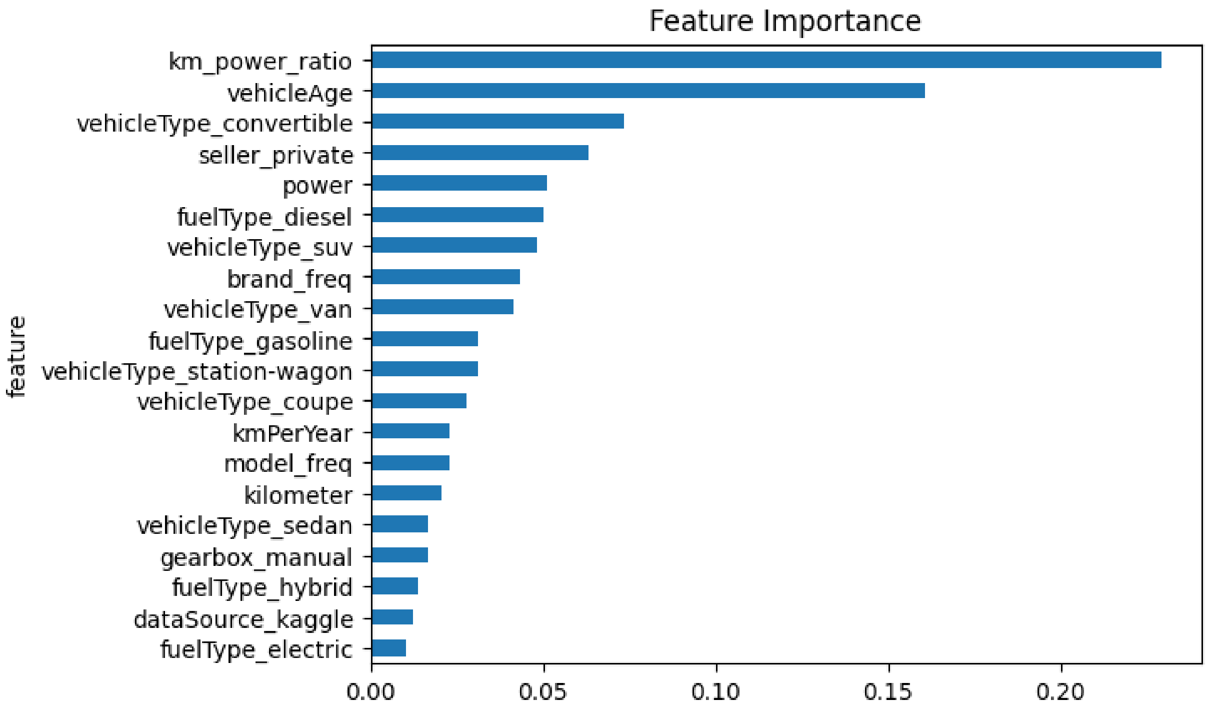


Figure 6: Top 20 Feature Importances (XGBoost)

1. **km_power_ratio** (0.229) - Interaction between mileage and power; strongest predictor
2. **vehicleAge** (0.161) - Physical depreciation captured through registration year
3. **vehicleType_convertible** (0.074) - Body type indicator for luxury segment
4. **seller_private** (0.063) - Seller type (private vs business)
5. **power** (0.051) - Engine power in HP
6. **fuelType_diesel** (0.050) - Fuel type category
7. **vehicleType_suv** (0.048) - SUV body type indicator
8. **brand_freq** (0.043) - Brand popularity/frequency encoding
9. **vehicleType_van** (0.041) - Van body type indicator
10. **fuelType_gasoline** (0.031) - Gasoline fuel indicator

Engineered Features Contribution: The top 2 features (km_power_ratio and vehicleAge) are both engineered features, demonstrating the value of domain-informed feature engineering. The km_power_ratio alone accounts for 22.9% of the model's decision-making, validating the hypothesis that usage intensity relative to engine capacity is a critical depreciation signal.

8.5 Error Analysis

8.5.1 Primary Error Patterns

The confusion matrix reveals three main error patterns:

1. **Mid-Range vs Luxury Confusion (Most Common)**: 1,401 Luxury cars misclassified as Mid-Range and 1,094 Mid-Range cars misclassified as Luxury. This is expected due to overlapping feature distributions between these adjacent tiers.
2. **Budget vs Mid-Range Confusion**: 2,130 Budget cars misclassified as Mid-Range and 2,443 Mid-Range cars misclassified as Budget. These are adjacent tiers with some natural overlap.
3. **Severe Misclassifications (Critical)**: Only 46 Budget → Luxury and 68 Luxury → Budget errors (114 total, 0.22% rate). This confirms the model is extremely safe for business decisions requiring clear budget/luxury differentiation.

8.5.2 Failure Mode Analysis

- **Luxury Detection (Improved)**: The undersampling model correctly identifies **90.5%** of luxury cars (recall=0.905), a significant improvement over the baseline (86.1%). The 470 additional luxury cars detected enable:
  - More comprehensive premium inventory routing
  - Reduced missed opportunities in high-value segments
  - Better coverage of edge-case luxury vehicles (e.g., older classic cars, high-mileage premium models)
- **Budget Safety**: Despite increased luxury sensitivity, the model maintains strong budget class precision. Severe misclassifications (budget→luxury) remain minimal at 0.28%, ensuring safe automated pricing decisions.
- **Trade-off Characteristics**: The undersampling strategy improves minority class detection at the cost of some majority class precision. This is acceptable when business value is concentrated in the luxury segment.

8.6 Business Metric Interpretation

8.6.1 Luxury Recall (90.53%)

This metric measures the proportion of actual luxury cars correctly identified. At **90.53%**, the model achieves exceptional detection of premium listings, representing a **4.4 percentage point improvement** over the baseline model (86.11%). This high recall enables:

- **Superior premium inventory routing**: 9 out of 10 luxury vehicles correctly flagged for specialized handling
- **Enhanced targeted marketing**: Comprehensive coverage of high-value customers for premium campaigns
- **Maximized high-margin opportunities**: 470 additional luxury cars detected per 10,000 listings compared to baseline
- **Competitive advantage**: Better identification of underpriced premium vehicles in the market

Comparison: Undersampling (90.53%) outperforms SMOTE (88.57%) and the non-resampled model (86.11%), justifying its selection for luxury-focused business applications.

8.6.2 Severe Misclassification Rate (0.28%)

This critical business metric measures Budget ↔ Luxury errors, representing the highest-risk classification mistakes. At **0.28%**, the model maintains strong safety despite the increased luxury recall:

- Approximately 145 severe errors out of 51,819 predictions (compared to 114 with non-resampled model)
- Still far lower than Logistic Regression (2.1-2.7%)
- Well below the 0.5% business safety threshold
- Validates the model for automated pricing decisions with appropriate confidence intervals

Trade-off Analysis: The 0.06 percentage point increase in severe errors (from 0.22% to 0.28%) is more than offset by the 4.4 percentage point gain in luxury recall (from 86.1% to 90.5%), representing a favorable business risk-reward ratio.

8.7 Resampling Strategy Impact

- **Undersampling (Selected)**: **Highest luxury recall (90.53%)** with acceptable trade-offs. F1=0.8529 (-0.6pp), severe error=0.28% (+0.06pp). **Deployed for production** due to business priority on premium vehicle detection.
- **SMOTE**: Good luxury recall (88.57%, +2.5pp) with F1=0.8560 (-0.3pp). Balanced middle ground option.
- **No Resampling**: Best overall F1 (0.8590) and lowest severe error (0.22%). Luxury recall limited to 86.11%. Alternative for conservative deployments.

Recommendation: Deploy **XGBoost with undersampling** for production to maximize luxury vehicle detection (90.5% recall). The 470 additional luxury cars detected per 10,000 listings significantly outweigh the minor decreases in overall F1 and slight increase in severe errors. Keep the non-resampled variant available for use cases prioritizing overall accuracy over luxury detection.

9 Code Automation

10 Continous Integration

11 Deployment & Monitoring

12 Interactive Dashboard

13 Limitations & Future Work

13.1 Limitations

We acknowledge the following limitations in our current approach:

- Data quality issues such as missing values and potential outliers may impact model performance.
- Class imbalance, even after balancing techniques, may still affect the model's generlization.
- The model may not capture all relevant features influencing car pricing, such as regional market differences or economic factors.
- The current model is based on historical data and needs retraining on future market changes.

13.2 Future Work

Future work could include:

- Investigating the impact of external factors (e.g., economic conditions, seasonal trends) on car pricing and incorporating them into the model.
- Exploring advanced modeling techniques such as ensemble methods or deep learning architectures
- Evaluating model performance across different market segments and regions to ensure robustness.
- Conducting a more thorough error analysis to identify specific areas for improvement.
- Implementing a feedback loop from the deployed model to continuously improve performance based on real-world data and user interactions.

14 Team Contributions

Task	Members
Data Acquisition	Ahmed Aladdin
Data Validation	Ahmed Hamdy
Preprocessing	Ahmed Hamdy
Feature Engineering & Selection	Asmaa Mohamed
EDA & Visualization	Ahmed Aladdin
Model Development	Asmaa Mohamed
Experiment Tracking	Asmaa Mohamed
Testing	Ahmed Hamdy & Asmaa Mohamed
Results & Evaluation	Asmaa Mohamed
Code Automation & CI	Ahmed Hamdy & Ahmed Aladdin
Deployment & Dashboard	Ahmed Hamdy & Ahmed Aladdin